How much did we really earn in August?

The exact dataset from the reel: two freshers, one question, two different answers. Practise **WHERE**, the filter that separates real revenue from everything else, and use Claude to understand every line instead of just copying it.

A **Fresher A · filters only the month**

₹14,67,430

counts every row: pending, cancelled and refunds too

B **Fresher B · adds one line** RIGHT

₹12,21,180

`AND status = 'paid'` : only money we received

INSIDE

01 The dataset: one orders table with a status

02 Set it up in two minutes

03 Warm-up: recreate the reel

04 Five beginner questions

05 Answer key with what to tell your manager

HOW TO USE THIS PACK

1. **Load the data** into MySQL Workbench (page 2).
2. **Try each question yourself first.** Stuck? Use the Claude prompt on the card, and ask it to explain, not just answer.
3. **Check the answer key** and compare your query line by line.

SKILLS YOU WILL PRACTISE

`WHERE` `= with text` `> ⩾ with numbers` `AND` `BETWEEN` `IN` `<>` `OR + ( )` `Prompting Claude`

01 · THE DATASET

One orders table, four kinds of order

`sales_db_02` holds an online store's orders from March to August 2026. The column that matters in this pack is **status**: not every row is money we actually received.

orders	2,250 rows
order_id INT · PRIMARY KEY	Unique ID for each order
customer_id INT	Who placed it (links to **customers**)
order_date DATE	When it was placed
amount DECIMAL(10,2)	Order value in ₹. **Refunds are negative.**
status VARCHAR(12)	paid, pending, cancelled or refund

WHAT EACH STATUS MEANS

paid	Money received. **This is revenue.** (1,885 rows)
pending	Ordered, not paid yet. Might never arrive. (177 rows)
cancelled	The order did not happen. (124 rows)
refund	Money we gave back, stored as a negative amount. (64 rows)

FIRST 5 ORDERS

ORDER_ID	CUSTOMER_ID	ORDER_DATE	AMOUNT	STATUS
30005	1026	2026-03-01	₹6,890	pending
30026	788	2026-03-01	₹2,310	cancelled
10418	214	2026-03-02	₹2,450	paid
10419	87	2026-03-02	₹12,990	paid
30022	1085	2026-03-02	₹4,490	cancelled

Also included: `customers` (customer_id, name, city · 1,240 rows). Not needed for these five questions.

02 · SET IT UP IN TWO MINUTES

MYSQL WORKBENCH (RECOMMENDED)

1. Open MySQL Workbench and connect to your local server.
2. **File → Open SQL Script...** and choose `sales_db_02.sql`.
3. Click the **lightning bolt** to run the whole script.
4. Right-click **Schemas → Refresh All**. `sales_db_02` appears with both tables.

GOOD TO KNOW

Three files come with this pack: `sales_db_02.sql`, `orders.csv` and `customers.csv`. They are also **attached inside this PDF** (paperclip panel in your PDF reader).

It uses its own database, `sales_db_02`, so it will not touch Pack 01's `sales_db`. Run `USE sales_db_02;` before your queries.

03 · WARM-UP

Recreate the reel

Run both freshers' queries. If you get the same two numbers as the video, your setup is perfect.

FRESHER A

```sql
SELECT SUM(amount) AS revenue
FROM orders
WHERE order_date BETWEEN '2026-08-01' AND '2026-08-31';
```

REVENUE
₹14,67,430

FRESHER B

```sql
SELECT SUM(amount) AS revenue
FROM orders
WHERE order_date BETWEEN '2026-08-01' AND '2026-08-31'
  AND status = 'paid';
```

REVENUE
₹12,21,180

WHERE DID THE ₹2,46,250 GO?

```sql
SELECT status,
       COUNT(*)    AS orders,
       SUM(amount) AS amount
FROM orders
WHERE order_date BETWEEN '2026-08-01' AND '2026-08-31'
GROUP BY status
ORDER BY amount DESC;
```

STATUS	ORDERS	AMOUNT
paid	431	₹12,21,180
pending	38	₹1,65,440
cancelled	27	₹1,12,090
refund	14	−₹31,280

THE INSIGHT FROM THE REEL

₹2,46,250

Fresher A counted every August row; Fresher B kept only paid orders. The gap is money that was pending, cancelled or already refunded. One line made the difference.

Why do refunds lower the total? They are stored as negative amounts, so `SUM` subtracts them. Fresher A's number mixes money in and money out.

WHAT EACH LINE OF FRESHER B'S QUERY DOES

`SELECT SUM(amount) AS revenue`	Adds up the amount of every row that survives the filter.
`FROM orders`	The table we read from.
`WHERE order_date BETWEEN … AND …`	Keeps only rows from 1 to 31 August (both days included).
`AND status = 'paid'`	A second condition: the row must also be paid. Both must be true.

04 · PRACTICE QUESTIONS

Five questions a manager would really ask

Each one uses **WHERE**, the skill from the reel, and each is a little harder than the last. Try it before you look at the answer key.

Q1 WHERE = TEXT

LEVEL ● ○ ○

How many orders are still pending?

YOUR MANAGER ASKS

"How many orders are still waiting for payment, and how much money is stuck there?"

YOUR TASK

Return **one row**: the number of **pending** orders and their **total amount**, across all six months.

HINT

Text goes inside single quotes: `status = 'pending'`. `COUNT(*)` counts the rows that pass the filter, `SUM(amount)` adds them up.

ASK CLAUDE THE RIGHT WAY

```
I'm using MySQL. Table orders(order_id, customer_id, order_date, amount, status).
status is one of paid, pending, cancelled, refund; refund amounts are negative. Write
a query that counts the pending orders and adds up their amount. Explain why
'pending' needs quotes and what WHERE does, like I'm a beginner.
```

Q2 WHERE WITH NUMBERS

LEVEL ● ○ ○

Which are our biggest orders?

YOUR MANAGER ASKS

"Show me every order of ₹15,000 or more, biggest first."

YOUR TASK

Return **order_id, customer_id, order_date, amount** and **status** for every order of 15,000 or more, highest amount first. How many are there, and are they all paid?

HINT

Numbers need no quotes: `amount >= 15000`. `>=` means "greater than or equal to", so an order of exactly ₹15,000 is included. Finish with `ORDER BY amount DESC`.

ASK CLAUDE THE RIGHT WAY

```
I'm using MySQL. Table orders(order_id, customer_id, order_date, amount, status).
status is one of paid, pending, cancelled, refund; refund amounts are negative. List
every order with amount 15000 or more, biggest first. Explain the difference between
> and >= with an example.
```

Q3 AND + BETWEEN

LEVEL ● ● ○

What did we really earn in July?

YOUR MANAGER ASKS

"Same question as the reel, but for July. And this time, only real money."

YOUR TASK

Return July's revenue counting **paid orders only**. Then run it again **without** the status line. How far off would Fresher A have been?

HINT

Join two conditions with `AND`: the date range `order_date BETWEEN '2026-07-01' AND '2026-07-31'` and `status = 'paid'`. A row only counts when both are true.

ASK CLAUDE THE RIGHT WAY

```
I'm using MySQL. Table orders(order_id, customer_id, order_date, amount, status).
status is one of paid, pending, cancelled, refund; refund amounts are negative. Give
me July 2026 revenue from paid orders only. Explain how AND combines two conditions
and why BETWEEN includes both end dates.
```

Q4 IN

LEVEL ● ● ○

Where did the rest of August go?

YOUR MANAGER ASKS

"Fresher A's August number was ₹2.46 lakh too high. Which orders make up that gap?"

YOUR TASK

Return the **total amount** of August orders that are **pending, cancelled or refund**. Check that it matches the gap between the two freshers' answers. **Bonus:** get the same number using `<>` (not equal).

HINT

`IN` checks a list in one go: `status IN ('pending', 'cancelled', 'refund')`. It means the same as three conditions joined with `OR`, but it is shorter and harder to get wrong.

ASK CLAUDE THE RIGHT WAY

```
I'm using MySQL. Table orders(order_id, customer_id, order_date, amount, status).
status is one of paid, pending, cancelled, refund; refund amounts are negative. Add
up the August 2026 orders whose status is pending, cancelled or refund. Show me the
version with IN and the version with <>, and explain why both give the same answer
here.
```

Q5 AND VS OR · THE BRACKET TRAP

LEVEL ● ● ●

Why is my colleague's number so big?

YOUR MANAGER ASKS

"A colleague says pending plus cancelled orders in August come to ₹8.94 lakh. That sounds far too high. Can you check?"

YOUR TASK

Run the colleague's query below, then fix it. What is the right August number, and why was the first one wrong?

THE QUERY

```
SELECT COUNT(*) AS orders, SUM(amount) AS amount
FROM orders
WHERE status = 'pending' OR status = 'cancelled'
  AND order_date BETWEEN '2026-08-01' AND '2026-08-31';
```

HINT

SQL does `AND` before `OR`, the way maths does × before +. Without brackets, the date range only sticks to `cancelled`. Put brackets around the OR part, or use `IN`.

ASK CLAUDE THE RIGHT WAY

```
I'm using MySQL. Table orders(order_id, customer_id, order_date, amount, status).
status is one of paid, pending, cancelled, refund; refund amounts are negative. Here
is a query: WHERE status = 'pending' OR status = 'cancelled' AND order_date BETWEEN
'2026-08-01' AND '2026-08-31'. Why does it return pending orders from other months?
Show me how to fix it.
```

BEFORE YOU OPEN THE ANSWER KEY

1. **Read your result like a manager.** Does the number make sense? Would you put your name on it?
2. **Ask Claude to review your query**, not write it: "Here is my query and its result. Is it correct? What could go wrong?"
3. **Write one sentence** you would send your manager. The answer key has one for every question.

Q1 How many orders are still pending?

```sql
SELECT COUNT(*)    AS pending_orders,
       SUM(amount) AS pending_amount
FROM orders
WHERE status = 'pending';
```

PENDING_ORDERS	PENDING_AMOUNT
177	₹7,81,660

TELL YOUR MANAGER

We have 177 orders still waiting for payment, worth ₹7,81,660. That money is not in the bank yet, so it should not be counted as revenue.

Q2 Which are our biggest orders?

```sql
SELECT order_id, customer_id,
       order_date, amount, status
FROM orders
WHERE amount >= 15000
ORDER BY amount DESC;
```

TELL YOUR MANAGER

There are 12 orders of ₹15,000 or more, worth ₹2,10,880 together, and all of them are paid. Customer 917 placed 3 of them alone. The biggest unpaid order is only ₹7,110, so checking the status column confirms these are real money.

ORDER_ID	CUSTOMER_ID	ORDER_DATE	AMOUNT	STATUS
10544	377	2026-03-21	₹19,960	paid
10701	265	2026-04-10	₹19,530	paid
11056	29	2026-05-20	₹19,340	paid
10518	917	2026-03-17	₹18,460	paid
10829	917	2026-04-24	₹18,270	paid
11842	890	2026-07-29	₹18,120	paid
11244	917	2026-06-08	₹17,730	paid
11084	603	2026-05-22	₹17,060	paid
11772	458	2026-07-23	₹15,870	paid
12085	603	2026-08-15	₹15,740	paid
10780	342	2026-04-19	₹15,540	paid
10767	214	2026-04-17	₹15,260	paid

12 rows, all paid.

Q3 What did we really earn in July?

```sql
-- real money only
SELECT SUM(amount) AS july_revenue
FROM orders
WHERE order_date BETWEEN '2026-07-01' AND '2026-07-31'
  AND status = 'paid';
```

JULY_REVENUE
₹11,86,770

```sql
-- Fresher A's way: every row
SELECT SUM(amount) AS july_all_rows
FROM orders
WHERE order_date BETWEEN '2026-07-01' AND '2026-07-31';
```

JULY_ALL_ROWS
₹13,91,960

TELL YOUR MANAGER

July's real revenue was ₹11,86,770. Counting every row would have reported ₹13,91,960, which is ₹2,05,190 too high. Same mistake as Fresher A, one month earlier.

Q4 Where did the rest of August go?

```sql
SELECT SUM(amount) AS not_received
FROM orders
WHERE order_date BETWEEN '2026-08-01' AND '2026-08-31'
  AND status IN ('pending', 'cancelled', 'refund');
```

NOT_RECEIVED
₹2,46,250

```sql
-- bonus: same answer with <>
SELECT SUM(amount) AS not_received
FROM orders
WHERE order_date BETWEEN '2026-08-01' AND '2026-08-31'
  AND status <> 'paid';
```

NOT_RECEIVED
₹2,46,250

TELL YOUR MANAGER

₹2,46,250 of August's rows were never money we kept: ₹1,65,440 pending and ₹1,12,090 cancelled, less ₹31,280 already refunded. That is exactly the gap between ₹14,67,430 and ₹12,21,180.

IN or <>? Here they match because there are only four statuses. If a new status like `on_hold` appears tomorrow, `<> 'paid'` would quietly include it. `IN` says exactly what you mean.

Q5 Why is my colleague's number so big?

```sql
-- the colleague's query (wrong)
SELECT COUNT(*) AS orders, SUM(amount) AS amount
FROM orders
WHERE status = 'pending' OR status = 'cancelled'
  AND order_date BETWEEN '2026-08-01' AND '2026-08-31';
```

ORDERS	AMOUNT
204	₹8,93,750

```sql
-- fixed: brackets around the OR
SELECT COUNT(*) AS orders, SUM(amount) AS amount
FROM orders
WHERE (status = 'pending' OR status = 'cancelled')
  AND order_date BETWEEN '2026-08-01' AND '2026-08-31';
```

ORDERS	AMOUNT
65	₹2,77,530

```sql
-- also right, and easier to read
SELECT COUNT(*) AS orders, SUM(amount) AS amount
FROM orders
WHERE status IN ('pending', 'cancelled')
  AND order_date BETWEEN '2026-08-01' AND '2026-08-31';
```

ORDERS	AMOUNT
65	₹2,77,530

TELL YOUR MANAGER

The colleague's query returns 204 orders and ₹8,93,750 because SQL runs AND before OR: the August filter only applied to cancelled orders, so pending orders from every month got in. With brackets (or IN) the real August figure is 65 orders, ₹2,77,530.

HOW SQL READ THE COLLEAGUE'S QUERY

```
status = 'pending'
OR (status = 'cancelled' AND
August)
```

AND is done first, so the August filter only applies to cancelled orders. All 177 pending orders, from every month, get in.

Solved all five?

Comment the question you found hardest on the reel, and follow **analytixai_** on Instagram for more beginner SQL with AI.